

Apuntes – Semana 7: Redes Neuronales

Inteligencia Artificial

Mariela Solano Gómez – 2022437963

Fecha	09/04/2026
Profesor	Steven Pacheco Portuguez
Universidad	Instituto Tecnológico de Costa Rica
Semestre	I Semestre

Abstract—La clase inició con un repaso de la sesión anterior sobre preprocesamiento de datos, cubriendo los cinco pasos principales: *Data cleaning*, *Data integration*, *Data reduction*, *Data transformation* y *Data discretization*. Contenido evaluado en el quiz del martes 14 de abril. Posteriormente, se introdujo el tema de Redes Neuronales, abordando conceptos fundamentales como la clasificación MNIST, Regresión Logística binaria y multinomial, representación *one-hot*, y la matriz de pesos W .

I. NOTICIAS DE LA SEMANA

Anthropic anunció el lanzamiento de **Claude Mythos Preview**, un nuevo modelo de lenguaje de propósito general. Junto con este lanzamiento, se presentó el **Project Glasswing**, una iniciativa orientada a fortalecer la seguridad de sistemas de uso comercial masivo mediante la detección de vulnerabilidades.

Para ello, Anthropic estableció alianzas con diversas organizaciones, entre ellas Linux Foundation, Microsoft y Apple, a quienes se les ofrece acceso prioritario al modelo. La estrategia de divulgar los hallazgos primero a grandes compañías resulta acertada, ya que permite corregir vulnerabilidades antes de que puedan afectar a usuarios finales.

Más información: <https://www.anthropic.com/glasswing>

Adicionalmente, Anthropic anunció una reducción en la tasa de uso de Claude, lo que implica que los usuarios contarán con un menor límite de solicitudes disponibles.

II. CLASE ANTERIOR: PREPROCESAMIENTO DE DATOS

Tenemos distintos escenarios que afectan la calidad de los datos, entre ellos incompletitud, inexactitud o ruido e inconsistencia. Es por esto que necesitamos realizar un preprocesamiento de los datos para usarlos en nuestros modelos. Las principales tareas de preprocesamiento son:

II-A. Data Cleaning

Eliminación de ruido, corrección de inconsistencias, tratamiento de valores faltantes.

Valores faltantes

- Ignorar las tuplas con valores faltantes (peor de los escenarios).
- Completar manualmente.
- Usar un valor global constante.

- Rellenar con estadísticos (media, mediana, moda).
- Inferir valores mediante modelos estadísticos o de ML (regresión, k-NN, árboles de decisión).

Noisy Data (Regresión)

- Ajustar una función matemática a los datos para suavizar el ruido.
- Puede ser lineal o no lineal.

Noisy Data (Series Temporales)

- Aplicar técnicas de filtrado para suavizar fluctuaciones.
- *Media móvil*: promedio de los últimos n días para suavizar los datos.

II-B. Data Integration

Combinación de datos de múltiples fuentes heterogéneas en un repositorio coherente.

Redundancia

- Duplicación de información: no necesariamente valores idénticos, sino atributos que reflejan la misma información.
- Ejemplo: peso y altura están correlacionados; a mayor altura, generalmente mayor peso.

Se tienen dos técnicas para detectar redundancia:

Prueba χ^2 (datos nominales)

Mide la asociación entre variables categóricas comparando frecuencias observadas $O_{i,j}$ vs. esperadas $E_{i,j}$:

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{i,j} - E_{i,j})^2}{E_{i,j}}$$

- Grados de libertad: $(r - 1)(c - 1)$ para una tabla $r \times c$.
- Nivel de significancia α a escogencia; generalmente $\alpha = 0,05$.
- Si $\chi^2_{\text{calc}} \leq \chi^2_{\alpha, df}$: variables independientes; en caso contrario, asociadas.

Correlación (datos numéricos)

- **Pearson**: fuerza y dirección de la relación lineal; sensible a outliers; requiere distribución normal.
- **Spearman**: detecta relaciones monótonas; más robusto ante outliers.

II-C. Data Reduction

Reducción de volumen mediante selección de atributos, reducción de dimensionalidad o discretización.

II-D. Data Transformation

Normalización, estandarización, agregación y construcción de nuevas variables.

- **Min-Max Normalization:** escala los datos al rango $[0, 1]$:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- **Z-score:** distribución con media 0 y desviación estándar 1:

$$x' = \frac{x - \mu}{\sigma}$$

donde μ es la media y σ la desviación estándar.

II-E. Data Discretization

Transformación de atributos continuos en categóricos (intervalos). **Binning:** dividir un atributo continuo en intervalos (*bins*) y asignar cada valor a uno de ellos.

III. REDES NEURONALES

MNIST es un dataset clásico con 60,000 imágenes de dígitos escritos a mano (Fig. 1). La tarea consiste en identificar qué número representa cada imagen. Las imágenes originalmente son de 128×128 píxeles, reducidas a $28 \times 28 = 784$ features.

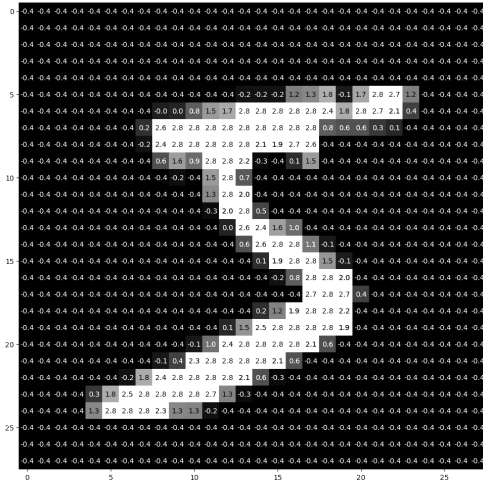


Fig. 1: MNIST Dataset.

Hasta ahora hemos trabajado con clasificación binaria, pero MNIST implica clasificar entre 10 clases (0–9). Antes de abordar eso, veamos cómo alimentar una imagen a una regresión logística.

De imagen a vector: Flatten

Una imagen no puede ingresarse directamente; hay que transformarla en un vector de características mediante **Flatten**: se concatenan todas las filas de píxeles en un solo vector (Fig. 2).

Imagen $28 \times 28 \xrightarrow{\text{Flatten}}$ vector de 784 entradas

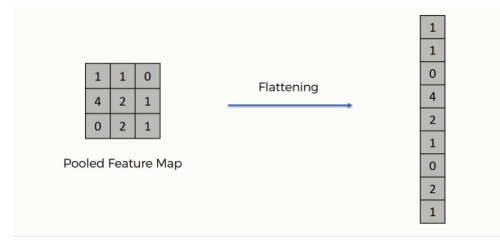


Fig. 2: Operación Flatten

A este conjunto de entradas se le denomina **Input Layer**. Para MNIST tiene 784 nodos (uno por píxel). La regresión logística recibe entonces:

- **Entradas:** vector de 784 características (píxeles).
- **Pesos:** uno por cada entrada.
- **Bias:** término independiente.
- **Función de activación:** sigmoide

Como se observa en la Fig. 3, esta estructura escala directamente desde el Input Layer hasta la neurona de salida.

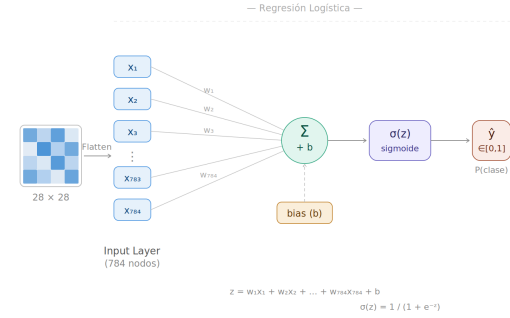


Fig. 3: Regresión logística con Input Layer de 784 entradas.

Limitación: para imágenes pequeñas esto es manejable, pero con imágenes grandes la cantidad de entradas crece demasiado, haciendo ineficiente a la regresión logística binaria.

III-A. Regresión Logística Multinomial

La regresión binaria solo responde si un dígito *es* o *no es* un 5. Para MNIST necesitamos distinguir entre las 10 clases, por lo que se extiende el modelo a la **Regresión Logística Multinomial**: en lugar de un único valor binario, el modelo produce un vector de probabilidades, una por clase.

Las etiquetas se representan con un **one-hot vector**: longitud K , con un 1 en la posición de la clase correcta y 0 en las demás. Para $K = 10$, el dígito 3 es:

$$\mathbf{y} = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$$

La Fig. 4 ilustra el concepto con tres clases alimenticias. El one-hot vector asigna 1 únicamente a la clase correcta:

- Hot dog $\rightarrow [1, 0, 0]$
- Borscht $\rightarrow [0, 1, 0]$
- Shawarma $\rightarrow [0, 0, 1]$

Solo una posición está “encendida” (*hot*), de ahí el nombre *one-hot*.

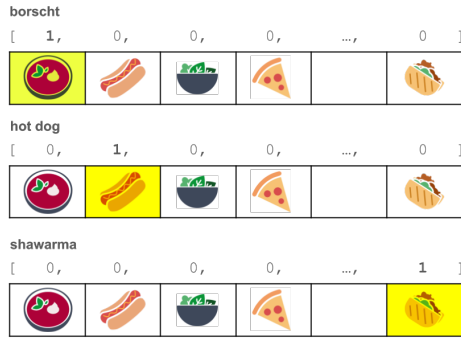


Fig. 4: One-hot encoding con tres clases.

1 feature (1 píxel)

El caso mínimo (Fig. 5): una sola entrada, 10 pesos y 10 neuronas de salida, una por clase. Reducido al absurdo, pero ilustra la estructura base.

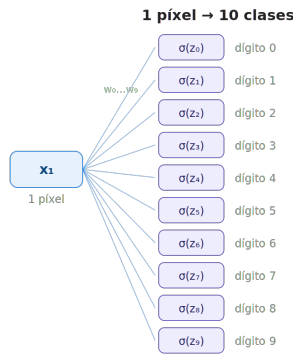


Fig. 5: Regresión logística multinomial con 1 feature.

2 features (2 píxeles)

Al agregar un segundo píxel (Fig. 6), cada neurona de salida recibe 2 pesos y un bias. La estructura se mantiene, pero el modelo dispone de más información.

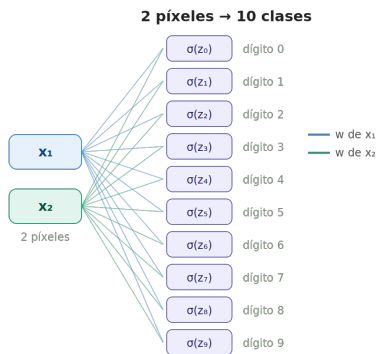


Fig. 6: Regresión logística multinomial con 2 features.

5 features (5 píxeles)

Con 5 píxeles (Fig. 7) ya tenemos $5 \times 10 = 50$ pesos, lo que empieza a asemejarse a los inicios de una red neuronal. A

esta estructura (cada entrada conectada a todas las neuronas de salida) se le denomina **Dense Layer** o **Fully Connected Layer**.

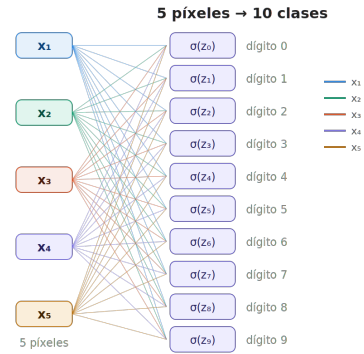


Fig. 7: Regresión logística multinomial con 5 features (Dense / Fully Connected Layer).

III-B. Conversión de vector a matriz

En lugar de calcular cada regresión logística individualmente (un ciclo de N operaciones), se puede expresar todo como una única multiplicación matricial usando álgebra lineal.

Todos los pesos se agrupan en la **matriz de pesos** W (Fig. 8), de dimensiones $j \times n$:

- j = número de clases (tamaño de la capa de salida).
- n = número de features (tamaño de la capa de entrada).

$w_{0,0}$	$w_{0,1}$	$w_{0,2}$	...	$w_{0,784}$
$w_{1,0}$	$w_{1,1}$	$w_{1,2}$	...	$w_{1,784}$
...	...	...	...	...
$w_{10,0}$	$w_{10,1}$	$w_{10,2}$	...	$w_{10,784}$

Fig. 8: Matriz de pesos W (cada fila es una regresión logística).

Para MNIST, W tiene dimensiones 10×784 : hay **10 regresiones logísticas dentro de una sola matriz**.

El bias se convierte en vector

En la regresión binaria el bias era un escalar b . Con j clases, cada regresión necesita su propio bias:

$$b \text{ (escalar)} \longrightarrow \mathbf{b} \text{ (vector de tamaño } j\text{)}$$

Para MNIST, $\mathbf{b}$ tiene tamaño 10. La operación completa, ilustrada en la Fig. 9, queda como una sola expresión:

$$\hat{Y} = \sigma(W \cdot \mathbf{x} + \mathbf{b})$$

donde $\mathbf{x}$ es el vector de píxeles (784×1), W la matriz de pesos (10×784), $\mathbf{b}$ el vector de bias (10×1) y $\hat{Y}$ el vector de probabilidades de salida (10×1).

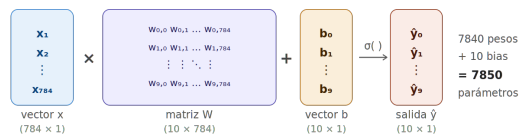


Fig. 9: Operación matricial completa: $\hat{Y} = \sigma(W \cdot \mathbf{x} + \mathbf{b})$.

Red neuronal de 1 capa para MNIST: la arquitectura descrita, compuesta por un Input Layer de 784 nodos y una capa densa de 10 neuronas de salida, constituye la forma más simple de una red neuronal aplicada a MNIST. En total cuenta con $784 \times 10 = 7840$ pesos + 10 bias = **7850** parámetros a ajustar mediante descenso del gradiente.